

- v7_regime Engine Implementation Plan
- File Structure
- Task 1: Directional indicators (ADX / +DI / -DI)
- Task 2: allowed_side() — the Layer 1 gate
- Task 3: Supertrend stop-and-reverse states
- Task 4: flip_states() — gate-constrained machine (the “never short a riser” guard)
- Task 5: Layer-1 backtest validation (walk-forward-lite + Sharpe)
- Task 6: Wire the v7_regime paper-trade engine
- Validation gate (before trusting v7_regime live)
- Out of scope (deliberately, per refuted findings)

v7_regime Engine Implementation Plan

For agentic workers: REQUIRED SUB-SKILL: Use superpowers:subagent-driven-development (recommended) or superpowers:executing-plans to implement this plan task-by-task. Steps use checkbox (– []) syntax for tracking.

Goal: Add a regime-gated long/short/flip paper-trading engine (v7_regime) that refuses longs in down-regimes and all directional trades in chop — the fix for today’s LONG_IN_BEAR bleed.

Architecture: Two pure, unit-tested layers built first, then wired into a v5-modeled engine. **Layer 1** (regime_gate.py) reads daily bars → emits allowed_side ∈ {LONG_ONLY, SHORT_ONLY, BOTH, FLAT} from ADX/DMI + SMA50 slope. **Layer 2** (supertrend_flip.py) reads intraday bars → a Supertrend stop-and-reverse state machine that proposes LONG/SHORT, then is **constrained** by Layer 1’s allowed_side (this is where “never short a riser” is enforced). A backtest harness validates Layer 1 alone before any engine wiring. The engine reuses v5’s pooled-state structure so the existing trade-audit.py picks it up automatically (it reads docs/paper-trades/{engine}/{date}.json).

Tech Stack: Python 3, pandas, numpy, pytest. Reuses prototype/v4/ml_engine.py (_atr, ADX math) and v5’s regime-module registry pattern.

Source spec: docs/research/2026-06-08_long-short-flip-spec.md

File Structure

File	Responsibility
prototype/v7/__init__.py	package marker
prototype/v7/regime_gate.py	Layer 1 — directional_indicators() (ADX/ +DI/-DI), allowed_side()

File	Responsibility
prototype/v7/supertrend_flip.py	Layer 2 — supertrend() (SAR states), flip_states() (gate-constrained machine)
tests/v7/test_regime_gate.py	unit tests for Layer 1
tests/v7/ test_supertrend_flip.py	unit tests for Layer 2
scripts/v7-regime-backtest.py	WFO-lite validation of Layer 1 over historical CSVs
scripts/v7_regime-paper-trade.py	the engine (modeled on scripts/v5-paper-trade.py)
scripts/launch-market.sh:~ENGINES	register v7_regime (commented for A/B at first)

Params (ADX cutoffs, Supertrend multiplier, lookbacks) are **textbook defaults to be tuned via the backtest**, not shipped as-is (per spec validation section).

Task 1: Directional indicators (ADX / +DI / -DI)

prototype/v4/ml_engine.py:_adx computes +DI/-DI internally but only returns ADX. Layer 1 needs all three, so expose them in the new module.

Files: - Create: prototype/v7/__init__.py (empty) - Create: prototype/v7/regime_gate.py - Test: tests/v7/test_regime_gate.py



Step 1: Write the failing test

```
# tests/v7/test_regime_gate.py
import numpy as np, pandas as pd
from prototype.v7.regime_gate import directional_indicators

def _series(vals): return pd.Series(vals, dtype="float64")

def test_uptrend_has_plus_di_above_minus_di():
    # 60 strictly rising closes → +DI should dominate, ADX should be high
    n = 60
    close = _series([100 + i for i in range(n)])
    high = close + 1.0
    low = close - 1.0
    adx, pdi, mdi = directional_indicators(high, low, close,
        period=14)
    assert pdi.iloc[-1] > mdi.iloc[-1]
    assert adx.iloc[-1] > 25
```



Step 2: Run test to verify it fails

Run:
cd ~/Documents/tinker/projects/tradepilot && python3 -m pytest tests/v7/
test_regime_gate.py::test_uptrend_has_plus_di_above_minus_di -v
Expected: FAIL — ModuleNotFoundError: prototype.v7.regime_gate



Step 3: Write minimal implementation

```
# prototype/v7/regime_gate.py
"""Layer 1 – daily allowed-side regime gate (see docs/research/
2026-06-08_long-short-flip-spec.md)."""
import numpy as np
import pandas as pd

def _atr(high, low, close, period=14):
    prev_close = close.shift(1)
    tr = pd.concat([(high - low),
                    (high - prev_close).abs(),
                    (low - prev_close).abs()], axis=1).max(axis=1)
    return tr.rolling(period).mean()

def directional_indicators(high, low, close, period=14):
    """Return (adx, plus_di, minus_di) as pd.Series (Wilder
DMI)."""
    plus_dm = high.diff().clip(lower=0)
    minus_dm = (-low.diff()).clip(lower=0)
    plus_dm[plus_dm < minus_dm] = 0
    minus_dm[minus_dm < plus_dm] = 0
    atr = _atr(high, low, close, period)
    plus_di = 100 * (plus_dm.rolling(period).mean() /
                    atr.replace(0, np.nan))
    minus_di = 100 * (minus_dm.rolling(period).mean() /
                    atr.replace(0, np.nan))
    dx = 100 * ((plus_di - minus_di).abs() / (plus_di +
                    minus_di).replace(0, np.nan))
    adx = dx.rolling(period).mean()
    return adx, plus_di, minus_di
```



Step 4: Run test to verify it passes

Run: python3 -m pytest tests/v7/test_regime_gate.py -v Expected:
PASS



Step 5: Commit

```
git add prototype/v7/__init__.py prototype/v7/regime_gate.py
      tests/v7/test_regime_gate.py
git commit -m "feat(v7): directional indicators (ADX/+DI/-DI) for
      regime gate"
```

Task 2: `allowed_side()` — the Layer 1 gate

Files: - Modify: `prototype/v7/regime_gate.py` (append) - Test: `tests/v7/test_regime_gate.py` (append)



Step 1: Write the failing tests

```
# tests/v7/test_regime_gate.py (append)
from prototype.v7.regime_gate import allowed_side

def _df(closes):
    c = _series(closes)
    return pd.DataFrame({"High": c + 1.0, "Low": c - 1.0,
                        "Close": c})

def test_strong_uptrend_is_long_only():
    df = _df([100 + i for i in range(80)])
    assert allowed_side(df) == "LONG_ONLY"

def test_strong_downtrend_is_short_only():
    df = _df([200 - i for i in range(80)])
    assert allowed_side(df) == "SHORT_ONLY"

def test_flat_choppy_is_flat():
    # oscillate ±1 around 100 → low ADX → FLAT (stand aside)
    closes = [100 + (1 if i % 2 == 0 else -1) for i in range(80)]
    assert allowed_side(_df(closes)) == "FLAT"
```



Step 2: Run to verify they fail

Run: `python3 -m pytest tests/v7/test_regime_gate.py -v -k allowed or downtrend or flat` Expected: FAIL — `ImportError: cannot import name 'allowed_side'`



Step 3: Implement

```
# prototype/v7/regime_gate.py (append)
def allowed_side(daily, adx_trend=25.0, adx_chop=20.0,
                 sma_period=50, slope_lookback=5):
    """daily: DataFrame with High/Low/Close in chronological
        order.
        Returns one of LONG_ONLY / SHORT_ONLY / BOTH / FLAT.
```

```

ADX gates PERMISSION (non-negotiable: <chop => FLAT); +DI/-DI
+ SMA50 slope
give DIRECTION. This is the rule that stops longing fallers /
shorting risers.
"""
if daily is None or len(daily) < max(sma_period +
    slope_lookback, 30):
    return "FLAT"
adx, pdi, mdi = directional_indicators(daily["High"],
    daily["Low"], daily["Close"])
a, p, m = adx.iloc[-1], pdi.iloc[-1], mdi.iloc[-1]
if np.isnan(a) or a < adx_chop:
    return "FLAT"
sma = daily["Close"].rolling(sma_period).mean()
if np.isnan(sma.iloc[-1]) or np.isnan(sma.iloc[-1 -
    slope_lookback]):
    return "FLAT"
slope = sma.iloc[-1] - sma.iloc[-1 - slope_lookback]
bullish = (p > m) and (slope > 0)
bearish = (m > p) and (slope < 0)
if bullish:
    return "LONG_ONLY"
if bearish:
    return "SHORT_ONLY"
return "BOTH" if a >= adx_trend else "FLAT"

```



Step 4: Run to verify pass

Run: `python3 -m pytest tests/v7/test_regime_gate.py -v` Expected:
PASS (all)



Step 5: Commit

```

git add prototype/v7/regime_gate.py tests/v7/test_regime_gate.py
git commit -m
    "feat(v7): allowed_side regime gate - ADX permission + DI/
    SMA50 direction"

```

Task 3: Supertrend stop-and-reverse states

Files: - Create: `prototype/v7/supertrend_flip.py` - Test: `tests/v7/test_supertrend_flip.py`



Step 1: Write the failing test

```

# tests/v7/test_supertrend_flip.py
import pandas as pd
from prototype.v7.supertrend_flip import supertrend

```



```

    if close.iloc[i - 1] >= final_lower.iloc[i - 1]:
        final_lower.iloc[i] = max(lower.iloc[i],
                                final_lower.iloc[i - 1])

state = pd.Series(1, index=close.index, dtype="int64")
for i in range(1, len(close)):
    prev = state.iloc[i - 1]
    if prev == 1:
        state.iloc[i] = -1 if close.iloc[i] <
        final_lower.iloc[i] else 1
    else:
        state.iloc[i] = 1 if close.iloc[i] >
        final_upper.iloc[i] else -1
return state

```



Step 4: Run to verify pass

Run: `python3 -m pytest tests/v7/test_supertrend_flip.py -v` Expected: PASS



Step 5: Commit

```

git add prototype/v7/supertrend_flip.py tests/v7/
    test_supertrend_flip.py
git commit -m "feat(v7): Supertrend stop-and-reverse state engine"

```

Task 4: `flip_states()` — gate-constrained machine (the “never short a riser” guard)

Files: - Modify: `prototype/v7/supertrend_flip.py` (append) - Test: `tests/v7/test_supertrend_flip.py` (append)



Step 1: Write the failing tests

```

# tests/v7/test_supertrend_flip.py (append)
from prototype.v7.supertrend_flip import flip_states

def test_short_signal_under_long_only_becomes_flat():
    # Supertrend says SHORT (-1) but Layer 1 only allows LONG →
    # must be FLAT, never SHORT
    assert flip_states([-1], ["LONG_ONLY"]) == ["FLAT"]

def test_long_signal_under_short_only_becomes_flat():
    assert flip_states([1], ["SHORT_ONLY"]) == ["FLAT"]

def test_flat_regime_forces_flat():
    assert flip_states([1, -1], ["FLAT", "FLAT"]) == ["FLAT",
        "FLAT"]

```

```
def test_both_regime_follows_supertrend():
    assert flip_states([1, -1, 1], ["BOTH", "BOTH", "BOTH"]) ==
           ["LONG", "SHORT", "LONG"]
```



Step 2: Run to verify they fail

Run: `python3 -m pytest tests/v7/test_supertrend_flip.py -v -k flip or regime or flat` Expected: FAIL — ImportError: cannot import name 'flip_states'



Step 3: Implement

```
# prototype/v7/supertrend_flip.py (append)
def flip_states(supertrend_states, allowed_sides):
    """Constrain the Supertrend signal by Layer 1's allowed_side,
        per bar.
        supertrend_states: iterable of +1/-1. allowed_sides: iterable
        of
        LONG_ONLY/SHORT_ONLY/BOTH/FLAT. Returns list of LONG/SHORT/
        FLAT.

        This is the guard that makes shorting-a-riser / longing-a-
        faller impossible:
        a side the regime forbids collapses to FLAT.
    """
    out = []
    for s, allowed in zip(supertrend_states, allowed_sides):
        want = "LONG" if s > 0 else "SHORT"
        if allowed == "FLAT":
            out.append("FLAT")
        elif allowed == "LONG_ONLY":
            out.append("LONG" if want == "LONG" else "FLAT")
        elif allowed == "SHORT_ONLY":
            out.append("SHORT" if want == "SHORT" else "FLAT")
        else: # BOTH
            out.append(want)
    return out
```



Step 4: Run to verify pass

Run: `python3 -m pytest tests/v7/ -v` Expected: PASS (all Layer 1 + Layer 2 tests)



Step 5: Commit


```
git add prototype/v7/supertrend_flip.py tests/v7/
    test_supertrend_flip.py
git commit -m
    "feat(v7): gate-constrained flip machine – forbidden side
    collapses to FLAT"
```

Task 5: Layer-1 backtest validation (walk-forward-lite + Sharpe)

Validate the gate ALONE over history before wiring an engine, per the spec: *does refusing longs in down-regimes turn bear days non-negative?* Uses the same daily CSVs the regime detector reads (prototype/v5/regime_detector.py:_load).

Files: - Create: scripts/v7-regime-backtest.py



Step 1: Write the script

```
#!/usr/bin/env python3
"""Backtest Layer 1 alone: for each day, allowed_side() decides
    the side; we take
    the index's next-day return on the allowed side (LONG_ONLY=+ret,
    SHORT_ONLY=-ret,
    FLAT=0, BOTH=+ret). Reports annualised Sharpe vs buy-and-hold. NO
    look-ahead:
    allowed_side(t) uses bars up to and including t; return is t→t+1.

Usage: python3 scripts/v7-regime-backtest.py <daily_csv> [--adx-
    trend 25 --adx-chop 20]
"""
import sys
from pathlib import Path
import numpy as np
import pandas as pd

ROOT = Path(__file__).resolve().parent.parent
sys.path.insert(0, str(ROOT))
from prototype.v7.regime_gate import allowed_side

def run(csv_path, adx_trend=25.0, adx_chop=20.0):
    df = pd.read_csv(csv_path)
    df = df.rename(columns={c: c.capitalize() for c in
        df.columns})
    df = df[["High", "Low",
        "Close"]].dropna().reset_index(drop=True)
    rets, gated = [], []
    for t in range(60, len(df) - 1):
        side = allowed_side(df.iloc[t : t + 1], adx_trend, adx_chop)
        nxt = (df["Close"].iloc[t + 1] - df["Close"].iloc[t]) /
            df["Close"].iloc[t]
        if side in ("LONG_ONLY", "BOTH"):
            gated.append(nxt)
```

```

    elif side == "SHORT_ONLY":
        gated.append(-nxt)
    else:
        gated.append(0.0)
    rets.append(nxt)
g, b = np.array(gated), np.array(rets)
def sharpe(x):
    return (x.mean() / x.std() * np.sqrt(252)) if x.std() > 0
    else 0.0
print(f"{Path(csv_path).stem}: gated Sharpe={sharpe(g):.2f}
buy&hold Sharpe={sharpe(b):.2f} "
      f"gated cum={g.sum()*100:.1f}% b&h cum={b.sum()*100:.
1f}%")

if __name__ == "__main__":
    args = [a for a in sys.argv[1:] if not a.startswith("--")]
    run(args[0])

```



Step 2: Run against one real symbol CSV

Run: `python3 scripts/v7-regime-backtest.py $(ls prototype/**/data/*.csv 2>/dev/null | head -1 || find . -name '*.csv' -path '*data*' | head -1)` Expected: prints a line with gated vs buy&hold Sharpe. (Acceptance: on a bearish symbol, gated drawdown < buy&hold drawdown.)



Step 3: Commit

```

git add scripts/v7-regime-backtest.py
git commit -m
    "feat(v7): Layer-1 walk-forward-lite backtest (gated vs
    buy&hold Sharpe)"

```

Tuning gate: before live, sweep `adx_trend/adx_chop` via walk-forward optimization (optimise in-sample, validate next out-of-sample window only) and report the Deflated Sharpe Ratio across the swept variants — do NOT pick the best in-sample params (spec validation section).

Task 6: Wire the v7_regime paper-trade engine

Build the engine by copying v5's structure (it's already pooled + long/short + regime-aware) and replacing its signal step with `allowed_side()` + `flip_states()`.

Files: - Create: `scripts/v7_regime-paper-trade.py` (start from `cp scripts/v5-paper-trade.py scripts/v7_regime-paper-trade.py`) - Modify: `scripts/launch-market.sh` (ENGINES array)



Step 1: Clone v5 and rename engine identity

```
cp scripts/v5-paper-trade.py scripts/v7_regime-paper-trade.py
```

In the new file, change the engine name constant (the value written as "engine" in `fresh_state()`, around `scripts/v5-paper-trade.py:239`) from "v5" to "v7_regime", and `confirm_state_file()` writes to `docs/paper-trades/v7_regime/{date}.json` (it derives from the engine name — verify the path).



Step 2: Add the regime modules to v5's lazy registry

v5 lazy-loads regime modules via a registry dict (`scripts/v5-paper-trade.py:110`, e.g. "regime": ("prototype.v5.regime_detector", "detect_regime")). Add entries:

```
"allowed_side": ("prototype.v7.regime_gate", "allowed_side"),
"supertrend":   ("prototype.v7.supertrend_flip",
                 "supertrend"),
"flip_states": ("prototype.v7.supertrend_flip",
                 "flip_states"),
```



Step 3: Replace the entry decision with the gate + flip

In the engine's per-stock signal step (where v5 currently decides direction), apply: `compute allowed = allowed_side(daily_bars[symbol]); compute Supertrend states on the intraday bars; position = flip_states(states, [allowed]*len(states)) [-1]`. Enter/flip only on LONG/SHORT; on FLAT close any open position for that symbol. Keep v5's sizing, pools, SL/target, and cost model unchanged. Add the intraday guard from the spec: skip if the stock is green-on-day for a SHORT or red-on-day for a LONG (VWAP/change_pct check already present in v5 signal rows).



Step 4: Smoke-test the engine imports + a dry scan

Run: `python3 -c "import ast; ast.parse(open('scripts/v7_regime-paper-trade.py').read()); print('parse OK')"` Then: `./scripts/sarathi-verify.sh --smoke --quiet; echo exit=$?` Expected: parse OK; smoke exit=0 (the launch gate that protects market open).



Step 5: Register in launch (commented, for opt-in A/B)

In `scripts/launch-market.sh` `ENGINES` array, add (commented so it doesn't auto-run until you opt in):

```
# "v7_regime|scripts/v7_regime-paper-trade.py" # regime-gated
long/short/flip (A/B vs v4/v5)
```



Step 6: Commit

```
git add scripts/v7_regime-paper-trade.py scripts/launch-market.sh
git commit -m
    "feat(v7): regime-gated long/short/flip paper engine (opt-
    in A/B)"
```

Once live, `scripts/trade-audit.py` picks up `v7_regime` automatically (it reads `docs/paper-trades/{engine}`), so the daily bear-day audit will A/B it against v4/v5 with zero extra wiring.

Validation gate (before trusting v7_regime live)

- All `tests/v7/` pass (`python3 -m pytest tests/v7/ -v`).
- Task 5 backtest: gated drawdown < buy&hold on a bearish symbol; ADX cutoffs tuned via WFO, DSR reported.
- One full paper session alongside v4/v5; `trade-audit.py` shows `v7_regime`'s `LONG_IN_BEAR` count ≈ 0 on a bear day.

Out of scope (deliberately, per refuted findings)

- FII/DII directional/threshold gating (all refuted) — exploratory only.
- Hurst as a hard switch (refuted) — optional soft hint in a later iteration.
- Mirror “-DI crosses below +DI” short rule (refuted) — shorts come from -DI > +DI in the gate.